17. Write an SQL statement to create a table named `Car`. The `Car` table should have the columns to hold a car's manufacturer, year model, and a 20-character vehicle ID number.
18. Write an SQL statement to delete the `Car` table you created in Algorithm Workbench 17.

Short Answer

1. If you are writing an application to store the customer and inventory records for a large business, why would you not want to use traditional text or binary files?
2. You hear a fellow classmate say the following: "JDBC is a standard language for working with database management systems. It was invented at IBM." Are these statements correct, or is he confusing JDBC with something else?
3. When we speak of database organization, we speak of such things as rows, tables, and columns. Describe how the data in a database is organized into these conceptual units.
4. What is a primary key?
5. What is a result set?
6. What are the relational operators in SQL for the following comparisons?

   Greater-than
   Less-than
   Greater-than or equal-to
   Less-than or equal-to
   Equal-to
   Not equal-to
7. What is the number of the first row in a table? What is the number of the first column in a table?
8. What is metadata? What is result set metadata? When is result set metadata useful?
9. What is a foreign key?

Programming Challenges

MyProgrammingLab™ *Visit www.myprogramminglab.com to complete many of these Programming Challenges online and get instant feedback.*

VideoNote
The Customer Inserter Problem

1. Customer Inserter

Write an application that connects to the `CoffeeDB` database, and allows the user to insert a new row into the `Customer` table.

2. Customer Updater

Write an application that connects to the `CoffeeDB` database, and allows the user to select a customer, then change any of that customer's information. (You should not attempt to change the customer number, because it is referenced by the `UnpaidOrder` table.)

3. Unpaid Order Sum

Write an application that connects to the `CoffeeDB` database, then calculates and displays the total amount owed in unpaid orders. This will be the sum of each row's `Cost` column.

4. Unpaid Order Lookup

Write an application that connects to the `CoffeeDB` database and displays a `JList` component. The `JList` component should display a list of customers with unpaid orders. When the user clicks on a customer, the application should display a summary of all the unpaid orders for that customer.

5. Population Database

Make sure you have downloaded the book's source code from the companion Web site at `www.pearsonhighered.com/gaddis`. In this chapter's source code folder you will find a program named `CreateCityDB.java`. Compile and run the program. The program will create a Java DB database named `CityDB`. The `CityDB` database will have a table named `City`, with the following columns:

Column Name	Data Type
`CityName` *Primary key*	`CHAR (50)`
`Population`	`DOUBLE`

The `CityName` column stores the name of a city and the `Population` column stores the population of that city. After you run the `CreateCityDB.java` program, the `City` table will contain 20 rows with various cities and their populations.

Next, write a program that connects to the `CityDB` database, and allows the user to select any of the following operations:

- Sort the list of cities by population, in ascending order.
- Sort the list of cities by population, in descending order.
- Sort the list of cities by name.
- Get the total population of all the cities.
- Get the average population of all the cities.
- Get the highest population.
- Get the lowest population.

6. `Personnel` Database Creator

Write an application that creates a database named `Personnel`. The database should have a table named `Employee`, with columns for employee ID, name, position, and hourly pay rate. The employee ID should be the primary key. Insert at least five sample rows of data into the `Employee` table.

7. Employee Inserter

Write a GUI application that allows the user to add new employees to the `Personnel` database you created in Programming Challenge 6.

8. Employee Updater

Write a GUI application that allows the user to look up an employee in the `Personnel` database you created in Programming Challenge 6. The user should be able to change any of the employee's information except employee ID, which is the primary key.

9. `PhoneBook` Database

Write an application that creates a database named `PhoneBook`. The database should have a table named `Entries`, with columns for a person's name and phone number.

Next, write an application that lets the user add rows to the `Entries` table, look up a person's phone number, change a person's phone number, and delete specified rows.